

# Стек и вызовы функций

# Повторение: стек

Стек – LIFO (last in, first out) структура данных. Аналогия – книжная стопка. Можем класть книгу только наверх стопки, достать можем только сверху стопки.

В данном курсе под стеком будем подразумевать аппаратный стек

Аппаратный стек – область памяти, выделяемая нашей программе операционной системой при запуске нашей программы.

# Повторение: стек

Стек – точно такая же память, к ней можно:

1. Обращаться по указателю
2. Делать push – класть значение на вершину стека
3. Делать pop – доставать значение с вершины стека

# Повторение: стек

Стек – точно такая же память, работать с ней можно так:

1. Обращаться по указателю
2. Делать push – класть значение на вершину стека
3. Делать pop – доставать значение с вершины стека

Есть специальный регистр, указывающий на вершину стека:

ESP – (stack-pointer)

Он изменяется при командах push и pop автоматически.

Если записать туда другое значение – стек “сломается”

# use-cases

В рукописных  
ассемблерных файлах  
стеком пользуются, чтобы  
сохранить регистры,  
уничтожаемые функцией  
перед ее вызовом

```
1 main:
2   push eax ; eax -> stack
3   push ecx ; ecx -> stack
4   push edx ; edx -> stack
5
6   call input_data ; register DESTROYER
7
8   ; Now stack looks like:
9   ; edx <- stack top
10  ; ecx
11  ; eax
12
13  pop edx ; mov top of the stak into edx, and remove top from stack
14
15  pop ecx ; mov top of the stak into ecx, and remove top from stack
16
17  pop eax ; mov top of the stak into ecx, and remove top from stack
18
19  xor eax, eax
20  ret
```

## use-cases

Регистры, которые должна сохранить вызывающая функция, называют **caller-saved**

Регистры, сохраняемые вызываемой функцией, называют **callee-saved**

*Назовите callee-, caller-saved регистры в примере*

```
1 main:
2   push eax ; eax -> stack
3   push ecx ; ecx -> stack
4   push edx ; edx -> stack
5
6   call input_data ; register DESTROYER
7
8   ; Now stack looks like:
9   ; edx <- stack top
10  ; ecx
11  ; eax
12
13  pop edx ; mov top of the stak into edx, and remove top from stack
14
15  pop ecx ; mov top of the stak into ecx, and remove top from stack
16
17  pop eax ; mov top of the stak into ecx, and remove top from stack
18
19  xor eax, eax
20  ret
```

## use-cases

Иногда, особенно в системном коде, нужно запустить вообще все регистры на стек. Для этого есть специальные инструкции:

- pusha ( push all)
- popa (pop all)

Конечно, можно не думать о callee- и caller-saved регистрах и всегда пользоваться инструкциями выше.

Но какие проблемы это вызовет?



**Рора (лат.) — род богомолов  
из семейства Deroplatyidae**

# Стековый фрейм

Рассмотрим пример.

- Где могли бы храниться переменные:  
data, data\_size, sum, min, max, i, v, range, avg?
- Какие плюсы и минусы у каждого типа хранилища?

```
1 int compute_stats() {  
2     static const int data[8] = {12, 7, 19, 3, 25, 14, 8, 10};  
3     static const int data_size = sizeof(data) / sizeof(data[0]);  
4  
5     int sum = 0;  
6     int min = data[0];  
7     int max = data[0];  
8  
9     for (int i = 0; i < data_size; i++) {  
10         int v = data[i];  
11  
12         sum += v;  
13  
14         if (v < min)  
15             min = v;  
16  
17         if (v > max)  
18             max = v;  
19     }  
20  
21     int range = max - min;  
22     int avg = sum / data_size;  
23  
24     return avg + range;  
25 }  
26
```



# Стековый фрейм

Рассмотрим пример.

- Где хранятся переменные:  
data, data\_size  
– **в секции .data**
- sum, min, max, i, v, range, avg  
– **на стеке**

```
1 int compute_stats() {
2     static const int data[8] = {12, 7, 19, 3, 25, 14, 8, 10};
3     static const int data_size = sizeof(data) / sizeof(data[0]);
4
5     int sum = 0;
6     int min = data[0];
7     int max = data[0];
8
9     for (int i = 0; i < data_size; i++) {
10         int v = data[i];
11
12         sum += v;
13
14         if (v < min)
15             min = v;
16
17         if (v > max)
18             max = v;
19     }
20
21     int range = max - min;
22     int avg = sum / data_size;
23
24     return avg + range;
25 }
26
```

# Стековый фрейм

## compute\_stats:

- sum, min, max, i, v, range, avg  
– **на стеке**

## fill\_buffer:

- value, step, i – **на стеке**

Но как тогда мы можем  
гарантировать, что другая  
функция не залезет в наш стек?

```
1 int fill_buffer() {  
2     int value = 10;  
3     int step = 3;  
4  
5     for (int i = 0; i < data_size; i++) {  
6         data[i] = value;  
7         value += step;  
8     }  
9  
10    return compute_stats();  
11 }
```

# Стековый фрейм

Но как тогда мы можем гарантировать, что другая функция не залезет в наш стек?

Можно зафиксировать за каждой функцией адреса стека, например:

- **compute\_stats**: 0x400 - 0x41C (28 байт, т.к. 7 переменных по 4 байт)
- **fill\_buffer**: 0x41C - 0x428 (12 байт, т.к. 3 переменных по 4 байт)

В чем минус такого решения?

# Стековый фрейм

На самом деле, есть два регистра, отвечающих за стек:

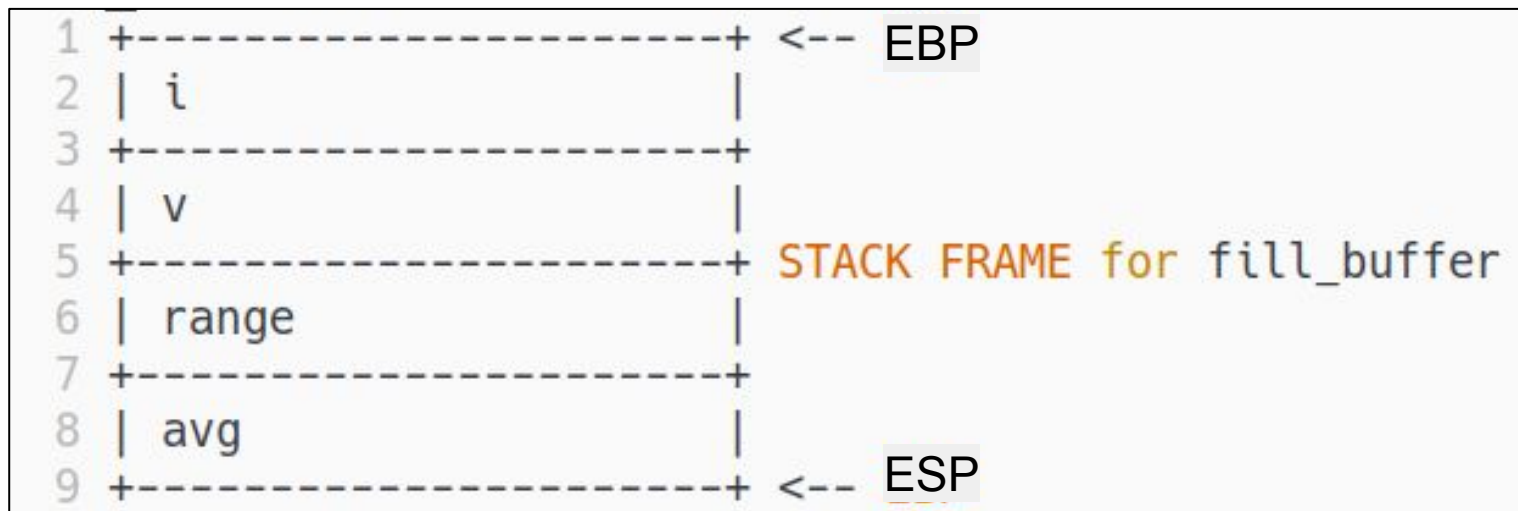
- ESP – stack pointer
- EBP – back pointer

Первый указывает на вершину стека, второй – на начало свободного пространства для следующей функции.

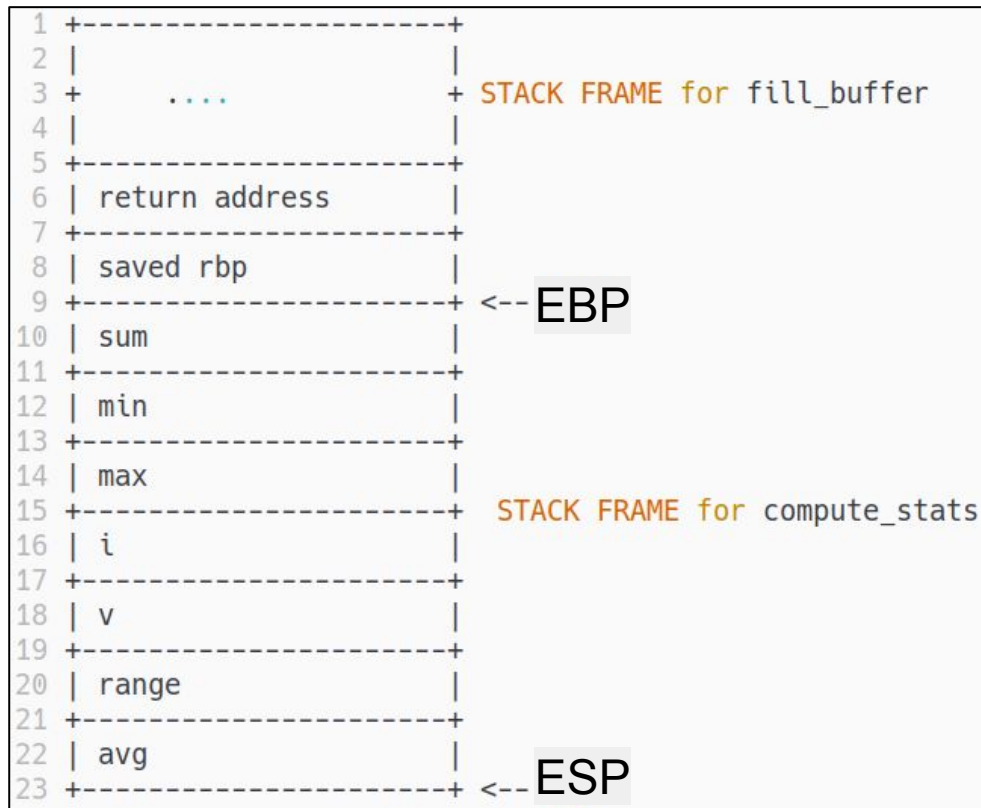
Пространство между RSP и RBP – стековый фрейм функции.

Это память на стеке, которой функция может распоряжаться и быть уверенной, что туда не залезет никто другой.

# Стековый фрейм



# Стековый фрейм. переделать картинку

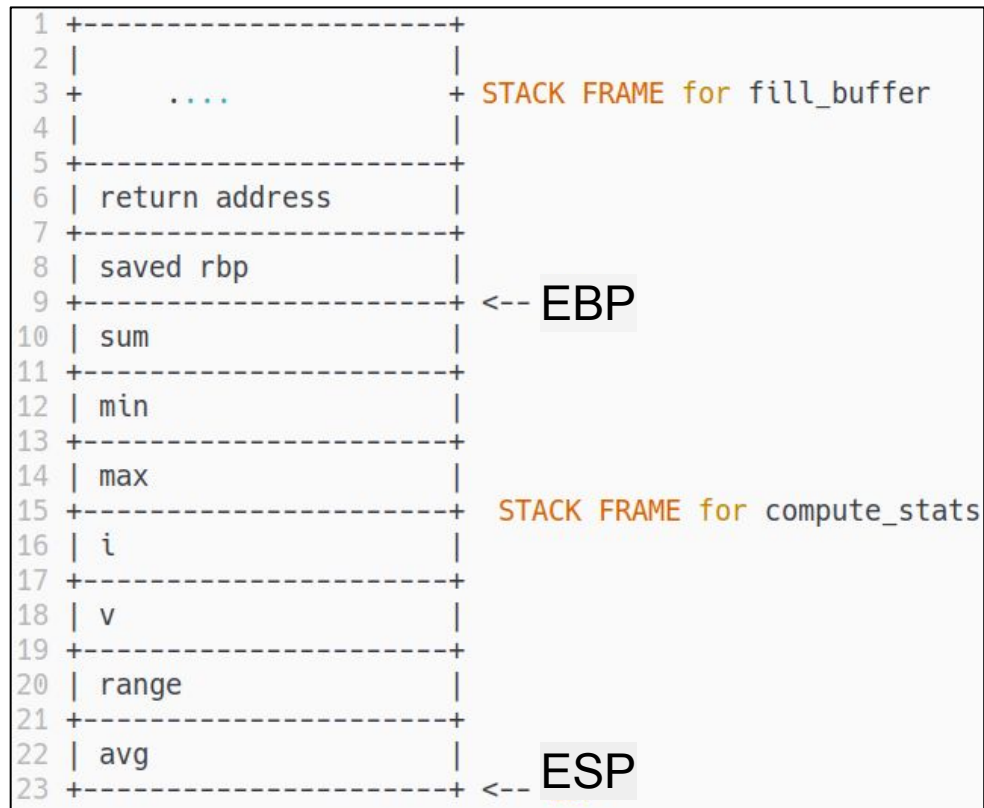


# Стековый фрейм

Стековый фрейм – договоренность

Компилятор генерирует код так,  
чтоб каждая функция не  
обращалась выше EBP

Однако, на высоких уровнях  
оптимизаций и в рукописном  
ассемблере это возможно



# Стековый фрейм

Начало функции, в которой аллоцируется стек называется **прологом**

Конец функции, в котором мы возвращаем все как было – **эпилогом**

Где на примере пролог и эпилог

```
1 prologue_example:
2     push ebp
3     mov  ebp, esp
4     sub  esp, 8           ; allocate 8 bytes on stack
5
6     mov  DWORD PTR [ebp-4], 5    ; int a = 5
7     mov  DWORD PTR [ebp-8], 7    ; int b = 7
8
9     mov  eax, [ebp-4]           ; eax = a
10    add  eax, [ebp-8]           ; eax = a + b
11
12    mov  esp, ebp
13    pop  ebp
14    ret
15
```



# Соглашения о вызовах

Зададимся вопросом, как передавать аргументы в функцию. Какие есть варианты?

# Соглашения о вызовах

Зададимся вопросом, как передавать аргументы в функцию. Какие есть варианты?

- Через регистры
- Через статические переменные
- Через стек

Представим, что мы хотим вызвать библиотечную функцию **printf**. Откуда мы узнаем, как именно программист передал в нее аргументы?

# Соглашения о вызовах

Представим, что мы хотим вызвать библиотечную функцию **printf**. Откуда мы узнаем, как именно программист передал в нее аргументы?

- Можно залезть в ассемблерный файл и посмотреть
- Можно прочесть документацию, если она есть
- Можно позвонить программисту и узнать

Это все не очень-то и удобно



# Соглашения о вызовах

Представим, что мы хотим вызвать библиотечную функцию **printf**. Откуда мы узнаем, как именно программист передал в нее аргументы?

Для решения этой проблемы были придуманы соглашения о вызовах. Это набор правил, по которым аргументы передаются в функцию.

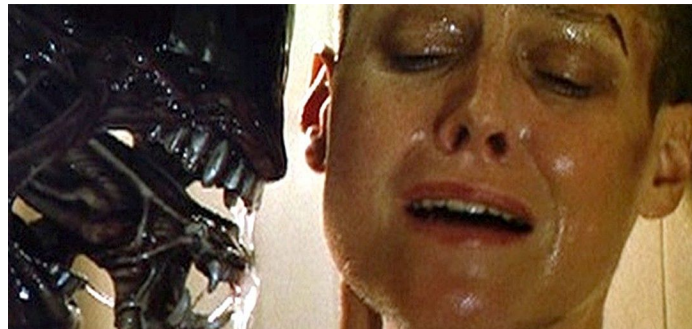
# Соглашения о вызовах

Ключевое слово здесь – “соглашения”. Мы уже успешно вызывали функции, не зная ничего о соглашениях о вызовах. Они пригождаются, когда мы хотим:

- Взаимодействовать с чужим кодом
- Чтоб чужие взаимодействовали с нашим кодом

Calling convention говорит нам:

- Кто чистит стек
- Как передаются аргументы
- Какие регистры сохраняет caller, а какие – callee



как ты передал аргументы в свою  
функцию

# Соглашения о вызовах

Существует множество соглашений о вызовах:

- stdcall
- fastcall
- cdecl
- System V AMD64 ABI
- thiscall

# Соглашения о вызовах. Cdecl

Cdecl – самое распространенное соглашение о вызовах в 32х битных программах под Unix. Компилятор генерирует такой код с флагом -m32

Не путать cdecl calling convention и cdecl определение функции (см. прошлый семестр)

- Аргументы передаются через стек **справа налево**
- Стек очищает caller
- Регистры EAX, ECX, EDX – caller-saved, остальные – callee-saved
- возвращаемое значение в EAX

# Соглашения о вызовах. System V ABI

System V ABI – самое распространенное соглашение о вызовах в 64х битных программах под Unix. Компилятор генерирует такой код по умолчанию

- Стек выделяет и очищает callee
- Возвращаемое значение в `eax`
- Регистры
  - RAX, RCX, RDX, RSI, RDI, R8, R9, R10, R11 – caller-saved,
  - RBX, RBP R12, R13, R14, R15 – callee-saved
- Аргументы:
  - первые 6 целочисленных передаются в RDI, RSI, RDX, RCX, R8, R9
  - если аргументов больше 6, остальные – через стек
  - floating-point аргументы через XMM-регистры



# Соглашения о вызовах. Магия за `io_*` функциями

Программист на C может управлять тем, какое соглашение о вызовах будет использовано. Для этого используются компиляторные директивы.

`io_*` функции – обычные обертки над `printf` и `scanf`, но со своим, кастомным, соглашением о вызовах

**`IO_ATTR`** – специальный макрос, в нем самое интересное

```
2 IO_ATTR  
3 void io_print_dec(int v)  
4 {  
5     printf("%d", v);  
6 }  
7
```

# Соглашения о вызовах. Магия за `io_*` функциями

**IO\_ATTR** – специальный макрос, в нем самое интересное

`regparm` – говорит компилятору использовать регистры для передачи параметров. Именно поэтому для `io_*` функций мы передаем аргументы в EAX, а не через стек, как говорит `cdecl` calling convention

```
2 #define IO_ATTR __attribute__((regparm(3),force_align_arg_pointer))
3
4 // Input facilities
5
6 // Read a 32-bit number using %d/%u/%x format, return value in EAX
7 IO_ATTR int      io_get_dec(void);
8 IO_ATTR unsigned io_get_udec(void);
9 IO_ATTR unsigned io_get_hex(void);
10
```

# Recap

- Вспомнили про стек
- Узнали про стековый фрейм
- Узнали про соглашения о вызовах
- Посмотрели как компилятор генерирует вызовы
- Увидели, что выделение стека – это одно вычитание из EBP
- Научились вызывать C-функции из ассемблера

# Рекомендуемая литература

- Bryant, R. E., & O'Hallaron, D. R. (2016). Computer Systems: A Programmer's Perspective (3rd ed.). Pearson. – **глава 3.7 стр 250 — 264**
- System V ABI — OSDev Wiki. URL: [https://wiki.osdev.org/System\\_V\\_ABI](https://wiki.osdev.org/System_V_ABI)
- Microsoft Docs. \_\_cdecl. URL: <https://learn.microsoft.com/en-us/cpp/cpp/cdecl?view=msvc-170>